Lecture on Pointers in C++ with Linked List Examples

Introduction to Pointers in C++

In C++, a **pointer** is a variable that holds the **memory address** of another variable. Pointers are essential in dynamic memory allocation, efficient array manipulation, and building advanced data structures like linked lists, trees, and graphs.

Basic Concepts of Pointers

1. **Declaring a Pointer:** To declare a pointer, use the `*` symbol after the data type. For example:

```
int *ptr;  // ptr is a pointer to an integer
```

2. **Assigning an Address to a Pointer:** You can assign the address of a variable to a pointer using the **address-of operator (`&`)**.

```
int x = 10;
ptr = &x;  // ptr now holds the address of x
```

3. **Dereferencing a Pointer:** The **dereference operator (`*`)** is used to access or modify the value that the pointer points to.

```
int y = *ptr;  // y is assigned the value of x (10)
```

Using Pointers as Parameters to Functions

Pointers are often used as parameters in C++ functions for two main reasons:

1. To **pass large objects efficiently** (without copying the entire object).
2. To **allow functions to modify** the actual value of a variable (pass by reference).

Pass-by-Value vs Pass-by-Reference

When a function is called in C++, the arguments are **copied** into the function parameters. If we want to modify the actual variables passed to the function, we can use pointers. Let's look at an example.

Example 1: Pass by Value

```
void increment(int num) {
    num++;  // Only modifies the local copy
```

```
}

int main() {
    int x = 5;
    increment(x);  // x remains 5
    cout << "x = " << x;  // Output: x = 5
    return 0;
}
```

In this case, the function `increment` operates on a **copy** of `x`, so the original value of `x` is unchanged.

Example 2: Pass by Pointer

Now, let's modify the actual value of `x` by passing a pointer to the function.

```
void increment(int *numPtr) {
    (*numPtr)++;  // Dereference the pointer to modify the actual variable
}

int main() {
    int x = 5;
    increment(&x);  // Pass the address of x
    cout << "x = " << x;  // Output: x = 6
    return 0;
}
```

In this example:

- We pass the **address of `x`** to the `increment` function (`&x`).
- Inside the function, the pointer `numPtr` holds the address of `x`, so we can modify `x` using `(*numPtr)`.

Key Points:

- **Passing by pointer** allows functions to modify the original variable.
- **Dereferencing** a pointer inside a function gives access to the original data.

Using Pointers in Dynamic Memory Allocation

Dynamic memory is allocated during runtime using the `new` keyword in C++.

Example: Dynamic Memory Allocation

```
int *ptr = new int;  // Dynamically allocate memory for an integer
*ptr = 42;           // Assign a value to the dynamically allocated memory
cout << *ptr;        // Output: 42
delete ptr;          // Free the dynamically allocated memory
```

- `new` allocates memory on the heap, and the pointer `ptr` holds the address of that memory.
- Always use `delete` to free dynamically allocated memory to avoid memory leaks.

Building a Linked List Using Pointers

A **linked list** is a dynamic data structure where each element (node) contains:

- **Data** (the actual information)
- A **pointer** to the next node in the list.

Linked lists are efficient for dynamic memory allocation, insertion, and deletion but require pointers to link nodes together.

Basic Structure of a Node

```
struct Node {
    int data;        // Stores data
    Node* next;      // Pointer to the next node
};
```

Here, each node contains an integer (`data`) and a pointer to the next node (`next`). If `next` is `nullptr`, it indicates the end of the list.

Step-by-Step Example: Building a Singly Linked List

Let's walk through creating a simple singly linked list with basic operations like insertion and traversal.

Step 1: Defining the Node Structure

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};
```

This `Node` struct will be used to create individual nodes in our linked list. Each node will store an integer (`data`) and a pointer to the next node (`next`).

Step 2: Inserting a Node at the End of the List

To insert a node at the end of the list, we need to traverse the list and find the last node, then link the new node to it.

```
void InsertAtEnd(Node** ptrToHeadPtr, int newData) {
   Node* newNode = new Node();
   newNode->data = newData;
   newNode->nextPtr = nullptr;

   if (*ptrToHeadPtr == nullptr) {
      *ptrToHeadPtr = newNode;
      return;
   }

   Node* curNodePtr = *ptrToHeadPtr;
   while (curNodePtr->nextPtr != nullptr) {
      curNodePtr = curNodePtr->nextPtr;
   }

   curNodePtr->nextPtr = newNode;
}
```

Step 3: Traversing the List

To print the values in the linked list, we traverse it from the head node to the end.

```
void PrintList(Node* headPtr) {
   Node* curNodePtr = headPtr;
   while (curNodePtr != nullptr) {
      cout << curNodePtr->data << " ";
      curNodePtr = curNodePtr->nextPtr;
   }
   cout << endl;

}
```

Step 4: Putting It All Together

Here's the full program that demonstrates inserting nodes into a linked list and printing the list:

```
#include <iostream>
using namespace std;

struct Node {
   int data;
   Node* nextPtr;
};

void InsertAtEnd(Node** ptrToHeadPtr, int newData) {
   Node* newNode = new Node();
```

```
    newNode->data = newData;
    newNode->nextPtr = nullptr;

    if (*ptrToHeadPtr == nullptr) {
        *ptrToHeadPtr = newNode;
        return;
    }

    Node* curNodePtr = *ptrToHeadPtr;
    while (curNodePtr->nextPtr != nullptr) {
        curNodePtr = curNodePtr->nextPtr;
    }

    curNodePtr->nextPtr = newNode;
}

void PrintList(Node* headPtr) {
    Node* curNodePtr = headPtr;
    while (curNodePtr != nullptr) {
        cout << curNodePtr->data << " ";
        curNodePtr = curNodePtr->nextPtr;
    }
    cout << endl;
}

int main() {
    Node* headPtr = nullptr;

    InsertAtEnd(&headPtr, 10);  // Insert 10 at the end
    InsertAtEnd(&headPtr, 20);  // Insert 20 at the end
    InsertAtEnd(&headPtr, 30);  // Insert 30 at the end
    cout << "Linked List: ";
    PrintList(headPtr);         // Output: 10 20 30

    return 0;

}
```

Explanation:

1. **insertAtEnd:** This function inserts a new node at the end of the list by:
    - Creating a new node.
    - Traversing to the last node.
    - Linking the new node to the last node's `next` pointer.
2. **printList:** This function traverses the list and prints the `data` of each node.
3. **main:** In the `main` function, we:
    - Start with an empty list (`head` is initially `nullptr`).
    - Insert three nodes with values 10, 20, and 30.
    - Print the contents of the list.

Conclusion

- **Pointers** are powerful tools in C++ that allow us to manipulate memory directly, pass variables by reference, and manage dynamic memory.
- They are crucial in implementing dynamic data structures such as **linked lists**.
- By passing pointers to functions, we can modify actual data and efficiently handle dynamic memory allocations.

Understanding pointers and their role in dynamic memory management is foundational for mastering C++ and working with advanced data structures.

```
// Function to delete the first node that contains the given value
void DeleteWithValue(Node **ptrToHeadPtr, int value) {
    if (*ptrToHeadPtr == nullptr) {
        cout << "List is empty." << endl;
        return;
    }

    Node* curNodePtr = *ptrToHeadPtr;

    // If the head node itself holds the value to be deleted
    if (curNodePtr != nullptr && curNodePtr->data == value) {
        *ptrToHeadPtr = curNodePtr->nextPtr; // Change head to the next node
        delete curNodePtr; // Free memory
        return;
    }

    // Search for the node that holds the value and keep track of the previous node
    Node* prev = nullptr;
    while (curNodePtr != nullptr && curNodePtr->data != value) {
        prev = curNodePtr;
        curNodePtr = curNodePtr->nextPtr;
    }

    // If value is not found in the list
    if (curNodePtr == nullptr) {
        cout << "Value not found in the list." << endl;
        return;
    }

    // Unlink the node from the linked list
    prev->nextPtr = curNodePtr->nextPtr;

    // Free memory
    delete curNodePtr;
}
```

A C++ Class version of the linkedlist

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node(int value) {
        data = value;
        next = nullptr;
    }
};

class LinkedList {
private:
    Node* head;

public:
    // Constructor
    LinkedList() {
        head = nullptr;
    }
```

```
// Destructor to clean up memory
~LinkedList() {
    clear();
}

// Copy constructor
LinkedList(const LinkedList& other) {
    head = nullptr;
    Node* current = other.head;
    while (current != nullptr) {
        insertAtEnd(current->data);
        current = current->next;
    }
}

// Assignment operator
LinkedList& operator=(const LinkedList& other) {
    if (this != &other) {
        clear();
        Node* current = other.head;
        while (current != nullptr) {
            insertAtEnd(current->data);
            current = current->next;
        }
    }
    return *this;
}

// Insert a node at the end of the list
void insertAtEnd(int newData) {
    Node* newNode = new Node(newData);
```

```
    if (head == nullptr) {
        head = newNode;
        return;
    }

    Node* current = head;
    while (current->next != nullptr) {
        current = current->next;
    }
    current->next = newNode;
}

// Insert a node at the beginning of the list
void insertAtBeginning(int newData) {
    Node* newNode = new Node(newData);
    newNode->next = head;
    head = newNode;
}

// Delete the first node that contains the given value
void deleteWithValue(int value) {
    if (head == nullptr) {
        cout << "List is empty." << endl;
        return;
    }
    // If the head node itself holds the value to be deleted
    if (head->data == value) {
        Node* temp = head;
        head = head->next;
        delete temp;
```

```
        return;
    }

    // Search for the node that holds the value and keep track of the previous node
    Node* current = head;
    Node* prev = nullptr;

    while (current != nullptr && current->data != value) {
        prev = current;
        current = current->next;
    }

    // If value is not found in the list
    if (current == nullptr) {
        cout << "Value not found in the list." << endl;
        return;
    }

    // Unlink the node from the linked list
    prev->next = current->next;
    delete current;
}

// Print the entire list
void printList() {
    Node* current = head;
    while (current != nullptr) {
        cout << current->data << " ";
        current = current->next;
    }
    cout << endl;
```

```
}

// Check if the list is empty
bool isEmpty() {
    return head == nullptr;
}

// Clear the entire list
void clear() {
    Node* current = head;
    while (current != nullptr) {
        Node* temp = current;
        current = current->next;
        delete temp;
    }
    head = nullptr;
}

// Get the size of the list
int getSize() {
    int count = 0;
    Node* current = head;
    while (current != nullptr) {
        count++;
        current = current->next;
    }
    return count;
}

// Search for a value in the list
bool contains(int value) {
```

```
        Node* current = head;
        while (current != nullptr) {
            if (current->data == value) {
                return true;
            }
            current = current->next;
        }
        return false;
    }
};

int main() {
    LinkedList list;

    // Insert nodes at the end
    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);

    cout << "Linked List after inserting 10, 20, 30: ";
    list.printList();  // Output: 10 20 30

    // Insert at beginning
    list.insertAtBeginning(5);
    cout << "After inserting 5 at beginning: ";
    list.printList();  // Output: 5 10 20 30

    // Delete a value
    list.deleteWithValue(20);
    cout << "After deleting 20: ";
    list.printList();  // Output: 5 10 30
```

```
    // Try to delete non-existent value
    cout << "Trying to delete 100: ";
    list.deleteWithValue(100);  // Output: Value not found in the list.

    // Check if list contains values
    cout << "List contains 10: " << (list.contains(10) ? "Yes" : "No") << endl;
    cout << "List contains 25: " << (list.contains(25) ? "Yes" : "No") << endl;

    // Get size
    cout << "List size: " << list.getSize() << endl;

    // Test copy constructor
    LinkedList list2 = list;
    cout << "Copied list: ";
    list2.printList();

    // Clear the list
    list.clear();
    cout << "After clearing: ";
    list.printList();
    cout << "Is list empty: " << (list.isEmpty() ? "Yes" : "No") << endl;

    // Test assignment operator
    LinkedList list3;
    list3 = list2;
    cout << "Assigned list: ";
    list3.printList();

    return 0;
}
```